Algorithmic Problem Collection

Selected independently designed algorithmic problems with complete editorials.

Timofei Gorchakov

Contents

1 Maximum Cylindrical Shift Score **2**

2 Palindromic Grid Paths **5**

3 Minimum XOR with Binary Tree Path Values **9**

4 Minimum Variance Reconstruction **15**

5 Two Targets on a Billiard Table **19**

6 Longest Path in a Prefix Graph **24**

Problem 1: Maximum Cylindrical Shift Score

You are given a cylindrical surface divided into n horizontal rings and m equally spaced vertical sectors. Each cell of the resulting $n \times m$ grid contains an integer.
Each horizontal ring may be rotated independently by any integer number of sectors. Rotating a ring cyclically moves all values in that ring, and different rings may be rotated by different amounts.
The values are given as an $n \times m$ matrix M, where row i lists the values on the i-th horizontal ring in cyclic order.
After rotating the rings, define the **score** of the cylinder as follows:

1. For each vertical sector, compute the sum of all values aligned in that sector.
2. Subtract the smallest sector sum from the largest sector sum.

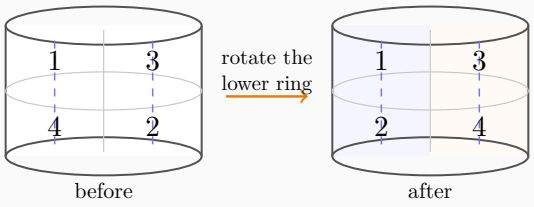
In other words, if the sector sums after all rotations are

$$S_0, S_1, \ldots, S_{m-1},$$

then

$$\text{score} = \max_{0 \le j < m} S_j - \min_{0 \le j < m} S_j.$$

Return the maximum possible score obtainable by rotating the horizontal rings.



Examples

Example 1
Input:

$$M = \begin{bmatrix} 1 & 3 \\ 4 & 2 \end{bmatrix}$$

Output: 4
Explanation:
The matrix represents two horizontal rings with two sectors each. Rotate the second ring by one sector. This changes the alignment to

$$\begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix}.$$

The two vertical sector sums are 3 and 7, so the score is

$$7 - 3 = 4.$$

No other choice of ring rotations produces a larger score.

Example 2
Input:

$$M = \begin{bmatrix} 1 & 3 & 6 \\ 4 & 2 & 5 \\ 9 & 7 & 8 \end{bmatrix}$$

Output: 9
Explanation:
Rotate the first horizontal ring to obtain

$$\begin{bmatrix} 6 & 1 & 3 \\ 4 & 2 & 5 \\ 9 & 7 & 8 \end{bmatrix}.$$

The vertical sector sums are 19, 10, and 16. Therefore,

$$\text{score} = 19 - 10 = 9.$$

No other choice of ring rotations produces a larger score.

Constraints

$$1 \leq n, m < 500$$

$$0 \leq M_{i,j} \leq 10^9$$

Solution

Rotating a horizontal ring by an integer number of sectors is exactly equivalent to cyclically shifting the corresponding row of M. Therefore, the cylindrical interpretation leads to the same algebraic operation on the input matrix.
Let the vertical sector sums after all rotations be

$$S_0, S_1, \ldots, S_{m-1}.$$

The score can be rewritten as

$$\max_j S_j - \min_k S_k = \max_{j,k}(S_j - S_k).$$

Therefore, instead of directly maximizing the difference between the largest and smallest sector sums, we can fix two sectors j and k and maximize $S_j - S_k$.
For a fixed pair of sectors, only their cyclic distance matters. Let

$$d = (k - j) \bmod m.$$

Consider a single horizontal ring i. After an appropriate rotation, some element $M_{i,p}$ will occupy sector j, while the element

$$M_{i,(p+d) \bmod m}$$

will occupy sector k.
Thus, for this ring, the maximum possible contribution to $S_j - S_k$ is

$$\max_{0 \le p < m} \left(M_{i,p} - M_{i,(p+d) \bmod m} \right).$$

Since every ring can be rotated independently, these choices can be optimized independently for every ring. Hence, for a fixed cyclic distance d, the maximum possible difference between the sums of the two sectors is

$$F(d) = \sum_{i=0}^{n-1} \max_{0 \le p < m} \left(M_{i,p} - M_{i,(p+d) \bmod m} \right).$$

Finally, we consider all possible cyclic distances:

$$\text{Answer} = \max_{0 \le d < m} F(d).$$

Expanding the definition of $F(d)$ gives

$$\boxed{\text{Answer} = \max_{0 \le d < m} \sum_{i=0}^{n-1} \max_{0 \le p < m} \left(M_{i,p} - M_{i,(p+d) \bmod m} \right)}$$

This can be computed in $O(nm^2)$ time and $O(1)$ additional space.

Complexity

- **Time:** $O(nm^2)$
- **Additional space:** $O(1)$

Problem 2: Palindromic Grid Paths

Let w be a string of length l consisting only of the characters `L`, `R`, `U`, and `D`.
Interpret w as a path on an infinite two-dimensional grid, starting at the cell with coordinates $(0, 0)$, where each character moves the path by one cell:

- `L` moves one cell to the left;
- `R` moves one cell to the right;
- `U` moves one cell up;
- `D` moves one cell down.

Given three integers x, y, and l, return the number of strings w satisfying all of the following conditions:

1. w has length exactly l;
2. the path encoded by w starts at $(0, 0)$ and ends at (x, y);
3. w is a palindrome.

Return the answer modulo

$$10^9 + 7.$$

Examples

Example 1
Input:

$$x = 2, \qquad y = 2, \qquad l = 4$$

Output: 2
Explanation:
The two valid strings are

$$\texttt{URRU}, \qquad \texttt{RUUR}.$$

Both are palindromes of length 4 and lead from $(0, 0)$ to $(2, 2)$.

Example 2
Input:

$$x = 0, \qquad y = 3, \qquad l = 7$$

Output: 9
Explanation:
The valid strings are

$$\begin{array}{lll} \texttt{RLUUULR}, & \texttt{LRUUURL}, & \texttt{DUUUUUD}, \\ \texttt{UDUUUDU}, & \texttt{LURURUL}, & \texttt{RULULUR}, \\ \texttt{UUDUDUU}, & \texttt{ULRURLU}, & \texttt{URLULRU}. \end{array}$$

Example 3
Input:

$$x = 1, \qquad y = 2, \qquad l = 4$$

Output: 0

Example 4
Input:

$$x = 0, \qquad y = 1, \qquad l = 3$$

Output: 1
Explanation:
The valid string is

$$\texttt{UDU}.$$

It is a palindrome of length 3 and leads from $(0, 0)$ to $(0, 1)$.

Constraints

$$0 \leq x, y \leq 10^9$$

$$1 \leq l \leq 10^5$$

Return the answer modulo $10^9 + 7$.

Solution

Let

$$k = \left\lfloor \frac{l}{2} \right\rfloor.$$

The first k moves of a palindromic string form an ordinary grid walk; suppose its displacement is (u, v). The second half is the reverse of the first half as a string, so it contains exactly the same moves and has the same displacement (u, v).
Therefore, if $l = 2k$ is even, then

$$(x, y) = 2(u, v).$$

If $l = 2k + 1$ is odd and the central move has displacement $c = (d_x, d_y)$, then

$$(x, y) = 2(u, v) + c.$$

Thus it remains to count ordinary length-k grid walks with a prescribed displacement.

Counting one half.
For nonnegative integers u and v, let $W(u, v, k)$ be the number of strings of exactly k moves whose net displacement is (u, v). If

$$k < u + v \qquad \text{or} \qquad k - u - v \text{ is odd},$$

then $W(u, v, k) = 0$. Otherwise, let

$$t = \frac{k - u - v}{2}.$$

The value t is the total number of cancelling move pairs. If exactly p of these pairs are horizontal, the move counts are

$$R = u + p, \qquad L = p, \qquad U = v + t - p, \qquad D = t - p.$$

For a fixed p, these counts sum to k, so the corresponding number of strings is a multinomial coefficient. Hence

$$\boxed{W(u, v, k) = \sum_{p=0}^{t} \frac{k!}{(u + p)!\, p!\, (v + t - p)!\, (t - p)!}} \pmod{10^9 + 7}.$$

Even length.
If $l = 2k$, both x and y must be even. If either coordinate is odd, the answer is 0. Otherwise,

$$\boxed{\text{answer} = W\left(\frac{x}{2}, \frac{y}{2}, k\right).}$$

Odd length.
If $l = 2k + 1$, consider each of the four possible central moves

$$c = (d_x, d_y) \in \{(1, 0), (-1, 0), (0, 1), (0, -1)\}.$$

For one fixed central move, the first-half displacement must be

$$u = \frac{x - d_x}{2}, \qquad v = \frac{y - d_y}{2}.$$

This move contributes only when both numerators are even and $u, v \geq 0$. Summing all valid contributions gives

$$\boxed{\text{answer} = \sum_{c \in \{R,L,U,D\}} W\left(\frac{x - d_x}{2}, \frac{y - d_y}{2}, k\right),}$$

where terms with nonintegral or negative arguments are omitted. By parity, only central moves along the uniquely odd coordinate can contribute, but considering all four moves directly is simpler and less error-prone.
Factorials and inverse factorials up to l can be precomputed modulo $10^9 + 7$. Each term of W is then evaluated in $O(1)$ time.

Complexity

- **Time:** $O(l)$
- **Additional space:** $O(l)$

Problem 3: Minimum XOR with Binary Tree Path Values

You are given the root of a binary tree. Each node contains either 0 or 1.
A **valid path** starts at any node and moves only downward through child edges until it reaches a leaf. A path may consist of a single node if that node is a leaf.
Read the node values along the path from its starting node to the leaf as the bits of a binary integer, with the starting node providing the most significant bit.
Leading zeroes are allowed. For example, the path

$$0 \to 0 \to 1 \to 1$$

represents

$$0011_2 = 3.$$

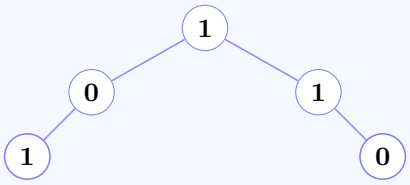
You are also given an integer array M.
For every M_i, return the minimum possible value of

$$M_i \oplus \text{value}(P),$$

where P may be any valid path and $\oplus$ denotes bitwise XOR.
Return an array containing the answer for every element of M.

Examples

Example 1
Input:



$$M = [3, 4, 5]$$

Output:

$$[1, 1, 0]$$

Explanation:
The distinct values represented by valid paths are

$$\{0, 1, 2, 5, 6\}.$$

For example,

$$1 \to 0 \to 1$$

represents $101_2 = 5$, while

$$1 \to 0$$

on the right branch represents $10_2 = 2$.
Therefore,

$$3 \oplus 2 = 1, \qquad 4 \oplus 5 = 1,$$

and

$$5 \oplus 5 = 0.$$

These are the minimum possible XOR values.

Example 2
Input:
The tree is a right-skewed chain with node values

$$0 \to 1 \to 1.$$

$$M = [0, 2, 7]$$

Output:

$$[1, 1, 4]$$

Explanation:
The three valid paths represent

$$011_2 = 3, \qquad 11_2 = 3, \qquad 1_2 = 1.$$

Thus the distinct available values are

$$\{1, 3\}.$$

The minimum XOR values are

$$0 \oplus 1 = 1, \qquad 2 \oplus 3 = 1,$$

and

$$7 \oplus 3 = 4.$$

Notice that removing a leading zero from a path does not change the represented integer.

Constraints

$$1 \leq \text{number of nodes} \leq 10^5$$

Each node value belongs to $\{0, 1\}$.

$$1 \leq |M| \leq 2 \cdot 10^5$$

$$0 \leq M_i \leq 10^6$$

The tree may be highly unbalanced.

Solution

At first, the number of valid paths may appear too large to process.
A tree with many leaves can contain far more than $O(n)$ paths that start at an arbitrary node and end at a leaf. Moreover, some of those paths may contain tens of thousands of bits.
However, the bound on the values in M implies that almost all of those paths are irrelevant.

1. Only the last 20 bits can matter

Since

$$10^6 < 2^{20},$$

every query value satisfies

$$M_i < 2^{20}.$$

Let

$$B = 20, \qquad U = 2^B.$$

Consider a fixed query value $x = M_i$.
Every tree contains at least one leaf. A path consisting only of that leaf is valid and represents either 0 or 1. Therefore, there always exists some valid path value y for which

$$x \oplus y < 2^{20}.$$

Consequently, a path whose value satisfies

$$y \geq 2^{20}$$

can never be optimal.
Indeed, x has no set bits at position 20 or above, so any such high bit of y remains set in

$$x \oplus y.$$

Hence

$$x \oplus y \geq 2^{20},$$

which is already worse than the value obtained from a one-node leaf path.
Therefore, every optimal path can be assumed to represent an integer smaller than 2^{20}.
Now consider a valid path containing more than 20 nodes but representing a value smaller than 2^{20}. Every bit before its last 20 bits must be zero.
For example, a path of the form

$$0 \to 0 \to 0 \to b_1 \to \cdots \to b_{20}$$

represents exactly the same integer as the shorter path

$$b_1 \to \cdots \to b_{20}.$$

The shorter sequence is also a valid path: it simply starts at a descendant of the original starting node and ends at the same leaf.
Thus:

$$\boxed{\text{For every query, an optimal path of length at most 20 exists.}}$$

This is the key observation.

2. Generate all relevant path values in one tree traversal

Every valid path ends at a leaf.
Therefore, every relevant path is a suffix of some root-to-leaf path, and we only need suffixes of length at most $B = 20$.
Traverse the tree from the root while maintaining:

- `mask`: the last at most 20 bits of the current root-to-node path;
- `len`: the number of available bits, capped at 20.

When visiting a node with value b, update

$$\texttt{mask} = ((\texttt{mask} \ll 1) \mid b) \,\&\, (2^{20} - 1).$$

Also update

$$\texttt{len} = \min(\texttt{len} + 1, 20).$$

Suppose the current node is a leaf.
The value of the path consisting only of this leaf is

$$\texttt{mask} \,\&\, (2^1 - 1).$$

The value of the path consisting of the leaf and its parent is

$$\texttt{mask} \,\&\, (2^2 - 1).$$

More generally, for

$$1 \le k \le \texttt{len},$$

the suffix of length k represents

$$y_k = \texttt{mask} \,\&\, (2^k - 1).$$

We represent the set of available values directly as the leaves of a complete binary trie of depth 20; with $U = 2^{20}$, value y corresponds to leaf $U + y$.
Whenever such a suffix value y_k is generated, directly mark

$$\texttt{nonempty}[U + y_k] = \texttt{true}.$$

Different paths may represent the same integer; they simply mark the same trie leaf again and require no special handling.
The DFS visits every tree node once and generates at most 20 suffix values at each leaf.

3. Build the internal trie information

Let

$$S \subseteq [0, 2^{20})$$

denote the values whose trie leaves were marked during the DFS.
The remaining problem is:

$$\min_{y \in S}(x \oplus y)$$

for every query x.
Represent every number using exactly 20 bits, adding leading zeroes when necessary.
A binary trie allows this minimum XOR value to be found greedily.
Because the universe contains exactly 2^{20} possible values, the trie can conveniently be represented as a complete binary tree of depth 20.
Let

$$U = 2^{20}.$$

Using heap-style indexing:

- node 1 is the root;
- the children of node v are $2v$ and $2v + 1$;
- the leaves $U, \ldots, 2U - 1$ correspond to the values $0, \ldots, U - 1$.

After the DFS is complete, process the trie bottom-up:

$$\texttt{nonempty}[v] = \texttt{nonempty}[2v] \vee \texttt{nonempty}[2v + 1].$$

Thus $\texttt{nonempty}[v]$ tells us whether the prefix represented by that trie node contains at least one available path value.

4. Answer a query greedily

To minimize

$$x \oplus y,$$

process the bits from most significant to least significant.
At a given bit position, we prefer y to have the same bit as x, because this produces a 0 in the XOR result.
If the corresponding trie branch contains at least one available value, take it.
Otherwise, we must choose the opposite bit, which produces a 1 in that position.
This greedy choice is optimal because a difference in a more significant bit outweighs every possible choice of all less significant bits combined.
For example, when processing bit b, producing 0 there is always better than producing 1, regardless of the lower b bits.
After 20 steps, the accumulated bits form exactly

$$\min_{y \in S}(x \oplus y).$$

Each query therefore takes

$$O(20)$$

time.

Correctness summary

We prove the algorithm in three steps.
First, every query value is smaller than 2^{20}, while a one-node leaf path always provides a candidate whose XOR with the query is also smaller than 2^{20}. Therefore, no path value at least 2^{20} can be optimal.
Second, any path longer than 20 nodes whose value is smaller than 2^{20} must begin with leading zeroes. Removing those leading zeroes produces a shorter valid path ending at the same leaf and representing the same integer. Hence an optimal answer always comes from a path of length at most 20.
The DFS enumerates exactly all such paths: every valid path ends at a leaf and is a suffix of its root-to-leaf path, while every suffix generated at a leaf is itself a valid downward path ending at that leaf.
Finally, the binary trie contains exactly the relevant path values. At every bit, choosing a branch matching the corresponding query bit minimizes the most significant still-undetermined bit of the XOR result whenever such a choice is possible. Therefore, the greedy trie traversal returns the minimum XOR over all relevant path values, which is also the minimum over all valid paths.

Implementation details

The tree may contain 10^5 nodes in one chain, so an iterative DFS is preferable in languages where deep recursion may overflow the call stack.
Each DFS state only needs:

- the current node;
- the current 20-bit mask;
- the current capped length.

For a leaf, enumerate

$$k = 1, \ldots, \texttt{len}$$

and compute

$$\texttt{mask} \,\&\, ((1 \ll k) - 1).$$

For $k = 20$, $1 \ll 20$ fits comfortably in ordinary integer types.
The implicit trie contains 2^{21} array positions up to a constant difference, approximately two million nodes. A byte or boolean per node is sufficient.
No path value larger than $2^{20} - 1$ ever needs to be constructed, even if the original tree has depth 10^5.

Complexity

Let

- n be the number of tree nodes;
- L be the number of leaves of the original tree;
- $q = |M|$;
- $B = 20$;
- $U = 2^{20}$.

The DFS visits all n nodes once and generates at most B suffix values at each of the L leaves:

$$O(n + BL).$$

Building the implicit trie takes

$$O(U).$$

All q queries take

$$O(Bq).$$

Therefore, **Time:** $O(n + BL + U + Bq)$. Since $L \leq n$ and $B = 20$ is fixed, this simplifies to

$$O(n + 2^{20} + q)$$

with a small constant factor.

Additional space: $O(U + n)$ in the worst case when an explicit DFS stack is included, plus the returned answer array.

Problem 4: Minimum Variance Reconstruction

You are given an integer array M of length $n-1$.
Return **any** array A of n positive integers such that

$$A_i + A_{i+1} = M_i$$

for every

$$0 \le i < n-1,$$

and the variance of A is as small as possible among all arrays satisfying these conditions.
The variance of an array A is

$$\mathrm{Var}(A) = \frac{1}{n}\sum_{i=0}^{n-1}(A_i - \mu)^2,$$

where

$$\mu = \frac{1}{n}\sum_{i=0}^{n-1} A_i$$

is its mean.
If several arrays have the minimum possible variance, return any of them.

Examples

Example 1
Input:

$$M = [4, 2, 8]$$

Output:

$$[3, 1, 1, 7]$$

Explanation: The adjacent sums are

$$3+1=4, \qquad 1+1=2, \qquad 1+7=8.$$

This is the only array of positive integers with the required adjacent sums, so it necessarily has minimum variance.

Example 2
Input:

$$M = [5, 5]$$

Output:

$$[2, 3, 2]$$

Explanation: Its adjacent sums are

$$2+3=5, \qquad 3+2=5.$$

This array has minimum possible variance. The array

$$[3, 2, 3]$$

has the same minimum variance and would also be a valid answer.

Constraints

$$1 \le |M| \le 10^5,$$

$$2 \le M_i \le 10^9.$$

It is guaranteed that at least one valid array of positive integers exists.

Solution

1. All valid arrays have one degree of freedom

Let

$$x = A_0.$$

Once x is fixed, every other element is determined:

$$A_1 = M_0 - x, \qquad A_2 = M_1 - A_1, \qquad \dots$$

Define a base sequence c_i by

$$c_0 = 0, \qquad c_{i+1} = M_i - c_i.$$

Then every valid reconstruction has the form

$$A_i = c_i + (-1)^i x = \begin{cases} c_i + x, & i \text{ even}, \\ c_i - x, & i \text{ odd}. \end{cases}$$

Thus the entire family of reconstructions has only one degree of freedom. Increasing x by one increases every even-indexed element by one and decreases every odd-indexed element by one.

2. Find the feasible interval

Every element of A must be positive. For an even index i,

$$c_i + x \geq 1 \quad \Longrightarrow \quad x \geq 1 - c_i.$$

For an odd index i,

$$c_i - x \geq 1 \quad \Longrightarrow \quad x \leq c_i - 1.$$

Therefore all valid values of x form one integer interval

$$L \leq x \leq R,$$

where

$$L = \max_{i \text{ even}} (1 - c_i), \qquad R = \min_{i \text{ odd}} (c_i - 1).$$

Because $c_0 = 0$, the lower bound already implies $x \geq 1$. The statement guarantees that $L \leq R$.

3. Separate even and odd positions

Let

$$E = \{i : i \text{ is even}\}, \qquad O = \{i : i \text{ is odd}\},$$

and let

$$p = |E| = \left\lceil \frac{n}{2} \right\rceil, \qquad q = |O| = \left\lfloor \frac{n}{2} \right\rfloor.$$

Define

$$C_E = \sum_{i \in E} c_i, \qquad C_O = \sum_{i \in O} c_i.$$

The means of the even-indexed and odd-indexed elements are

$$\mu_E(x) = \frac{C_E}{p} + x, \qquad \mu_O(x) = \frac{C_O}{q} - x.$$

Adding the same value to every element of a group does not change the variance inside that group. Therefore the variances within the two parity classes are independent of x; only the distance between their means changes.

The total variance decomposes as

$$\operatorname{Var}(A) = \frac{p}{n} \operatorname{Var}(E) + \frac{q}{n} \operatorname{Var}(O) + \frac{pq}{n^2} \big(\mu_E(x) - \mu_O(x)\big)^2.$$

The first two terms are constant. Hence minimizing the variance is equivalent to minimizing

$$\big(\mu_E(x) - \mu_O(x)\big)^2.$$

This remains valid when n is odd: the two groups may have different sizes.

4. Find the real optimum

Over the real numbers, the variance is minimized when the two parity-class means are equal:

$$\frac{C_E}{p} + x = \frac{C_O}{q} - x.$$

Thus

$$x^* = \frac{1}{2}\left(\frac{C_O}{q} - \frac{C_E}{p}\right) = \frac{pC_O - qC_E}{2pq}.$$

Let

$$U = pC_O - qC_E, \qquad V = 2pq.$$

Then

$$x^* = \frac{U}{V}.$$

5. Respect integrality and positivity

The actual x must be an integer in $[L, R]$. Since the objective is a convex quadratic with its minimum at x^*, an optimal integer is simply a feasible integer closest to x^*.

- If $U \le VL$, choose $x = L$.
- If $U \ge VR$, choose $x = R$.
- Otherwise, let $k = \lfloor U/V \rfloor$ and compare k with $k + 1$.

Compare the two candidates exactly using

$$|Vx - U|.$$

No floating-point arithmetic is necessary. If the candidates are equally close, either produces a minimum-variance answer.

6. Reconstruct the answer

Once the optimal x has been chosen, set

$$A_0 = x$$

and scan M, computing

$$A_{i+1} = M_i - A_i.$$

This produces the required array.

Correctness summary

Every valid reconstruction is determined uniquely by $A_0 = x$, and all feasible values of x form one integer interval.
Changing x translates all even-indexed values together in one direction and all odd-indexed values together in the opposite direction. The variance inside each parity class is therefore fixed, and the only variable part of the total variance is the squared distance between the two parity-class means.
The real optimum makes those means equal. Restricting this convex objective to the feasible integers means that a feasible integer nearest to the real optimum is optimal. The reconstructed array consequently has minimum possible variance.

Implementation details

The recurrence for c_i, the feasible bounds, and the parity sums can all be computed in one pass without storing the base sequence. Reconstruct the answer in a second pass.
Use sufficiently wide integer arithmetic for products such as

$$pC_O, \qquad qC_E, \qquad VL, \qquad VR.$$

In C++, `__int128` is a safe choice for these intermediate values.

Complexity

Time: $O(n)$ **Additional space:** $O(1)$, excluding the returned array.

Problem 5: Two Targets on a Billiard Table

You are given an axis-aligned rectangular billiard table with vertices

$$(0,0), \qquad (a,0), \qquad (a,b), \qquad (0,b).$$

A billiard ball starts at the point

$$S = (x_s, y_s).$$

Two distinct target points

$$P = (x_p, y_p), \qquad Q = (x_q, y_q)$$

are also located strictly inside the table. The three points S, P, Q are pairwise distinct.
The ball may be launched from S in any non-zero direction. It then moves forever at constant speed. Whenever it hits a vertical side of the table, the horizontal component of its velocity changes sign. Whenever it hits a horizontal side, the vertical component changes sign. If it hits a corner, both components change sign.
Return `true` if there exists an initial direction such that the ball passes through both P and Q at positive times. The targets may be visited in either order. Otherwise, return `false`.
All coordinates are integers.

Examples

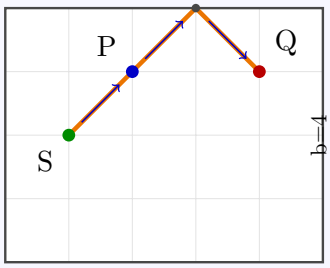
Example 1
Input:

$$a = 5, \qquad b = 4, \qquad \texttt{start} = [1,2], \qquad \texttt{p} = [2,3], \qquad \texttt{q} = [4,3].$$

Output: `true`
Explanation: Launch the ball in direction $(1,1)$. It follows the path

$$(1,2) \longrightarrow (2,3) \longrightarrow (3,4) \longrightarrow (4,3).$$

Thus it passes through P, reflects from the upper side, and then passes through Q.



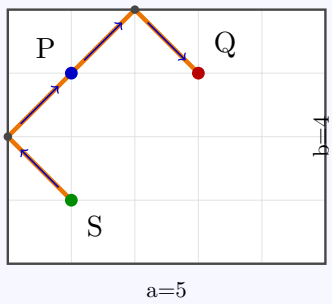
Example 2
Input:

$$a = 5, \qquad b = 4, \qquad \texttt{start} = [1, 1], \qquad \texttt{p} = [1, 3], \qquad \texttt{q} = [3, 3].$$

Output: `true`
Explanation: Launch the ball in direction $(-1, 1)$. It follows the path

$$(1, 1) \longrightarrow (0, 2) \longrightarrow (1, 3) \longrightarrow (2, 4) \longrightarrow (3, 3).$$

The ball reflects from the left side, passes through P, reflects from the upper side, and then passes through Q.



Example 3
Input:

$$a = 5, \qquad b = 4, \qquad \texttt{start} = [1, 1], \qquad \texttt{p} = [1, 2], \qquad \texttt{q} = [2, 1].$$

Output: `false`
Explanation: There is no initial direction whose trajectory passes through both target points.

Constraints

$$3 \le a, b \le 10^9,$$
$$0 < x_s, x_p, x_q < a,$$
$$0 < y_s, y_p, y_q < b.$$

The points S, P, Q are pairwise distinct.

Solution

1. Unfold the billiard table

Instead of reflecting the ball whenever it reaches a side, reflect the entire table. In the resulting infinite tiling of the plane, the trajectory becomes one straight ray starting at S.
For any point $X = (x_X, y_X)$, all of its reflected copies have coordinates

$$(\varepsilon_x x_X + 2ak,\ \varepsilon_y y_X + 2b\ell), \qquad \varepsilon_x, \varepsilon_y \in \{-1, 1\}, \qquad k, \ell \in \mathrm{Z}.$$

The signs are independent, so the copies of one target form four periodic classes. After translating the origin to S, a class of copies of X has the form

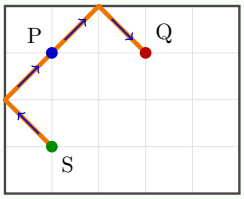
$$(\varepsilon_x x_X - x_s + 2ak,\ \varepsilon_y y_X - y_s + 2b\ell).$$

There are four classes for P and four for Q. Consequently, only

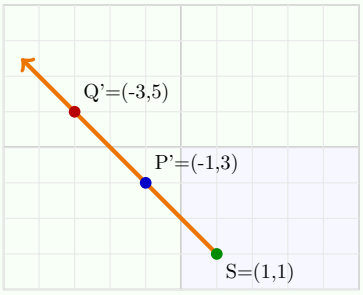
$$4 \cdot 4 = 16$$

pairs of classes have to be considered.

Reflected motion in the table



The same motion after unfolding



original table

Example 2 illustrates the transformation: the reflected trajectory becomes the straight line through

$$(1,1), \qquad (-1,3), \qquad (-3,5).$$

2. Work modulo the periods

Let

$$A = 2a, \qquad B = 2b.$$

Fix one of the 16 pairs of reflected classes. After translating S to the origin and reducing the coordinates modulo A and B, write its representatives as

$$p = (x_1, y_1), \qquad q = (x_2, y_2)$$

in the finite group

$$G = \mathrm{Z}_A \times \mathrm{Z}_B.$$

Why may we use an integer direction? If a successful straight ray reaches a reflected copy, its displacement from S has integer coordinates. The direction of that ray can therefore be scaled to a non-zero integer vector v.
Modulo the periods, this vector generates the cyclic subgroup

$$\langle v \rangle = \{0, v, 2v, 3v, \ldots\} \subseteq G.$$

A straight trajectory can reach representatives of both selected classes exactly when p and q belong to one common cyclic subgroup. This is equivalent to saying that

$$H = \langle p, q \rangle$$

is itself cyclic: a subgroup of a cyclic group is cyclic. Conversely, if H is cyclic, choose a generator g and an integer representative $v \in \mathrm{Z}^2$ of g. Since $p, q \in \langle g \rangle$, suitable integer multiples of v belong to the selected periodic classes, so v supplies a valid direction in the unfolded plane.

3. Compute the exponent of H

The order of an element is the smallest positive number of times it must be added to itself to obtain zero. For $(x, y) \in G$,

$$\mathrm{ord}(x, y) = \mathrm{lcm}\left(\frac{A}{\gcd(A, x)}, \frac{B}{\gcd(B, y)}\right).$$

Therefore, let

$$r_p = \operatorname{lcm}\left(\frac{A}{\gcd(A, x_1)}, \frac{B}{\gcd(B, y_1)}\right),$$

$$r_q = \operatorname{lcm}\left(\frac{A}{\gcd(A, x_2)}, \frac{B}{\gcd(B, y_2)}\right).$$

The exponent of a finite group is the smallest positive integer that sends every element to zero. Since H is generated by p and q,

$$E = \exp(H) = \operatorname{lcm}(r_p, r_q).$$

4. Compute the size of H

Consider the lattice $L \subseteq \mathrm{Z}^2$ generated by

$$(A, 0), \qquad (0, B), \qquad (x_1, y_1), \qquad (x_2, y_2).$$

Its index $d = [\mathrm{Z}^2 : L]$ is the greatest common divisor of all 2×2 determinants of these generators:

$$d = \gcd(AB,\ Ay_1,\ Ay_2,\ Bx_1,\ Bx_2,\ x_1y_2 - y_1x_2)\,.$$

Absolute values may be used inside the gcd. This gcd-of-minors identity is the two-dimensional form of Smith normal form.
The period lattice generated by $(A, 0)$ and $(0, B)$ has index AB. Adding p and q enlarges it to L, and its image modulo the period lattice is exactly H. Hence

$$\boxed{|H| = \frac{AB}{d}}.$$

5. Test whether H is cyclic

For a finite abelian group, the largest possible element order equals its exponent. It follows that

$$H \text{ is cyclic} \quad \Longleftrightarrow \quad |H| = \exp(H).$$

Indeed, a cyclic group has exponent equal to its size; conversely, if an element has order $|H|$, it generates all of H.
Thus the current pair of reflected classes is compatible exactly when

$$\boxed{\frac{AB}{d} = \operatorname{lcm}(r_p, r_q)}.$$

If this holds for at least one of the 16 pairs, return `true`. Otherwise, return `false`.

Why a forward ray is enough

No separate test is needed to distinguish a ray from a line. Suppose a residue class is reached as tv in G. Replacing t by

$$t + k\ \operatorname{ord}(v)$$

does not change the residue. By choosing k sufficiently large, the coefficient becomes positive. This can be done independently for both targets, so representatives of both classes lie on the same forward ray.

Implementation details

For each target, enumerate the four independent sign pairs $(\varepsilon_x, \varepsilon_y)$, normalize both coordinates to non-negative residues modulo A and B, and test all 16 pairs.
Use 64-bit integers. Since

$$A, B \le 2 \cdot 10^9, \qquad AB \le 4 \cdot 10^{18},$$

all final quantities fit in a signed 64-bit integer. Compute

$$\operatorname{lcm}(x, y) = \frac{x}{\gcd(x, y)}\, y$$

in this order to avoid an unnecessary intermediate overflow. A wider intermediate type may be used defensively for products and determinants.

Complexity

There are exactly 16 pairs of reflected classes, and each pair requires a constant number of gcd and lcm computations.

Time: $O(\log \max(a, b))$ **Additional space:** $O(1)$.

Problem 6: Longest Path in a Prefix Graph

You are given an array `words` of distinct strings.
Construct an undirected graph in which every string is a vertex. Two different vertices are connected by an edge if and only if one of their strings is a prefix of the other.
Return the maximum number of vertices in a simple path in this graph.
A simple path is a sequence of distinct vertices in which every two consecutive vertices are connected by an edge.

Examples

Example 1
Input:

`words = ["a", "ab", "abc", "abcd"]`

Output: 4
Explanation: Every shorter string is a prefix of every longer string, so every pair of vertices is connected. For example,

`"a"` ⟶ `"abc"` ⟶ `"ab"` ⟶ `"abcd"`

is a simple path containing all four vertices.

Example 2
Input:

`words = ["a", "ab", "abc", "ac", "acd", "ad", "ade"]`

Output: 5
Explanation: One longest path is

`"abc"` ⟶ `"ab"` ⟶ `"a"` ⟶ `"ac"` ⟶ `"acd"`.

It contains five vertices. The strings extending `"ab"`, `"ac"`, and `"ad"` form three different prefix branches. A simple path can pass through `"a"` only once, so it cannot use vertices from all three branches.

Example 3
Input:

`words = ["a", "ab", "abc", "b", "ba", "bac", "bad"]`

Output: 4
Explanation: The prefix graph has two connected components. The component containing `"a"` has a longest path of length 3. The other component contains the path

`"bac"` ⟶ `"ba"` ⟶ `"bad"` ⟶ `"b"`,

which has length 4. Therefore, the answer is 4.

Constraints

$$1 \le \texttt{words.length} \le 10^5,$$
$$1 \le \texttt{words[i].length} \le 10^3,$$
$$\sum_i |\texttt{words[i]}| \le 2 \cdot 10^5.$$

All strings in `words` are distinct and consist only of lowercase English letters.

Solution

The prefix graph may contain quadratically many edges. For example, in

```
["a", "ab", "abc", "abcd"]
```

every pair of vertices is adjacent. We therefore represent the prefix relation using a much smaller nearest-prefix forest.

1. Build a trie and the nearest-prefix forest

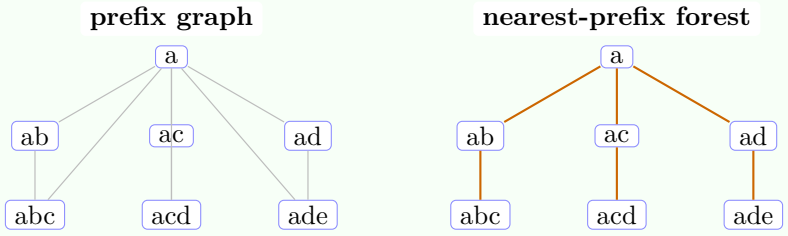
Insert all input strings into a trie and mark every trie node that corresponds to a complete input string.
For every input string v, define its parent to be the longest proper prefix of v that also appears in `words`. If no such prefix exists, v is a root. These parent relations form the **nearest-prefix forest** F.
Construct F during one traversal of the trie. Maintain the nearest marked ancestor on the current trie path. Whenever another marked node is reached, that ancestor is its parent in F. Thus F contains exactly `words.length` vertices and fewer than that many edges.

2. Relate the forest to the prefix graph

For two input strings u and v,

$$u \text{ is a prefix of } v \iff u \text{ is an ancestor of } v \text{ in } F.$$

The forward implication follows by repeatedly moving from v to its longest proper input prefix; this visits all input prefixes of v in decreasing order of length. The reverse implication follows because every parent edge of F connects a string to one of its prefixes.
Consequently, two vertices are adjacent in the original prefix graph exactly when one is an ancestor of the other in the nearest-prefix forest. The additional ancestor–descendant edges never need to be constructed.
The distinction is illustrated below for Example 2. The prefix graph on the left contains all ancestor–descendant edges; the nearest-prefix forest on the right retains only nearest-prefix parent edges.



3. Dynamic programming on the nearest-prefix forest

Let $dp[v]$ be the maximum number of vertices in a simple path whose vertices all lie in the forest subtree rooted at v.
Consider two different child subtrees of v. No vertex in one can be an ancestor of a vertex in the other, so the prefix graph has no edge between the two subtrees. A path that uses both must therefore pass through v. Because the path is simple, it can use vertices from at most two child subtrees of v.
On the other hand, v is adjacent in the prefix graph to every vertex in its forest subtree. If a child c contains an optimal path of length $dp[c]$, either endpoint of that entire path can be connected directly to v.
Let d_1 and d_2 be the two largest values among $dp[c]$ over all children c of v, using 0 when fewer than two children exist. Then

$$\boxed{dp[v] = 1 + d_1 + d_2}.$$

A path containing v cannot do better, because it uses at most two child subtrees. The bound is attainable by attaching optimal paths from the two selected subtrees to v. A path not containing v lies in a single child subtree and has length at most d_1, so it cannot improve the formula.
Since F may have several components, the final answer is

$$\boxed{\max_{r \text{ a root of } F} dp[r]}.$$

Correctness summary

The nearest-prefix forest preserves the prefix relation as ancestry, so it represents every adjacency of the prefix graph without explicitly storing all its edges. Different child subtrees of a forest vertex have no prefix graph edges between them, while the vertex itself is adjacent to every descendant. Therefore, an optimal path in its subtree consists of the vertex together with optimal paths from at most two child subtrees, exactly as described by the recurrence. Taking the maximum over all forest roots returns the longest simple path in the entire prefix graph.

Implementation details

Both the trie traversal and the forest DP may be implemented recursively. Their depth is at most the maximum input-string length, which is at most 10^3 under the constraints. An iterative traversal is also possible.

Complexity

Let

$$S = \sum_i |\texttt{words[i]}|.$$

Building and traversing the trie takes $O(S)$ time and space. The nearest-prefix forest contains $n = \texttt{words.length}$ vertices, and its DP takes $O(n)$ time. Since $n \le S$, the total complexity is

Time: $O(S)$ **Additional space:** $O(S)$.